

کتابچه آموزشی

از صفر تا درخت و گراف

تنظیم‌کننده: سینا شجاعیان

استاد: دکتر اسدی

دانشگاه آزاد اسلامی، واحد شیراز

مهندسی کامپیوتر — ترم ۱۴۰۵

© ۲۰۲۶ - کلیه حقوق این اثر متعلق به سینا شجاعیان است.

هرگونه کپی برداری، بازتولید، ویرایش، توزیع یا انتشار مجدد بخش یا تمام این اثر به هر شکل و در هر رسانه‌ای، بدون اجازه کتبی و صریح نویسنده ممنوع است. این نسخه صرفاً برای استفاده شخصی و مطالعه در نظر گرفته شده است.

مرجع رسمی و معتبر دانلود:

<https://github.com/3INA3>

SS-F24B-4128-445E-2026-DS2 :■■ ■■■■ ■■■■■■ ■■■

06-10-2026 :■■■■■ ■■■■

ساختمان داده • Python • الگوریتم • درخت • گراف

ساختمان داده

از صفر تا درخت و گراف

عنوان درس: [ساختمان داده (داده و ساختار)]

نام استاد: [دکتر اسدی]

تنظیم کننده: [سینا شجاعیان]

دانشگاه / دانشکده: [آزاد شیراز]

رشته و مقطع: [مهندسی کامپیوتر]

ترم / سال: [1405]

صف و صف حلقوی – اولین
ورودی، اولین خروجی
فصل ۳

پشته (Stack) – آخرین
ورودی، اولین خروجی
فصل ۲

تحلیل الگوریتم – چقدر طول
می کشد؟
فصل ۱

درخت – ساختار سلسله مراتبی
فصل ۶

لیست پیوندی – زنجیره ای از
نودها
فصل ۵

توابع بازگشتی – تابعی که
خودش را صدا می زند
فصل ۴

گراف – شبکه ی رئوس و یال ها
فصل ۷

قبل از شروع – ساختمان داده چیست؟

تصور کنید اتاق شما پر از وسایل است. اگر همه چیز را روی هم بریزید، پیدا کردن یک جوراب سخت می شود. اما اگر کمد داشته باشید، کتو داشته باشید، یا جعبه های مرتب، خیلی سریع تر پیدا می کنید. ساختمان داده دقیقاً همین است: روش های هوشمندانه برای چیدن داده در حافظه کامپیوتر تا بتوانیم سریع اضافه کنیم، حذف کنیم یا جست و جو کنیم.

۱. تحلیل الگوریتم – چقدر طول می کشد؟

آنالوژی: فرض کنید می خواهید یک کتاب را در کتابخانه پیدا کنید. اگر کتاب ها به هم ریخته باشند، ممکن است مجبور شوید همه ی قفسه ها را یکی یکی نگاه کنید (کند). اگر کتاب ها بر اساس حروف الفبا مرتب باشند، خیلی

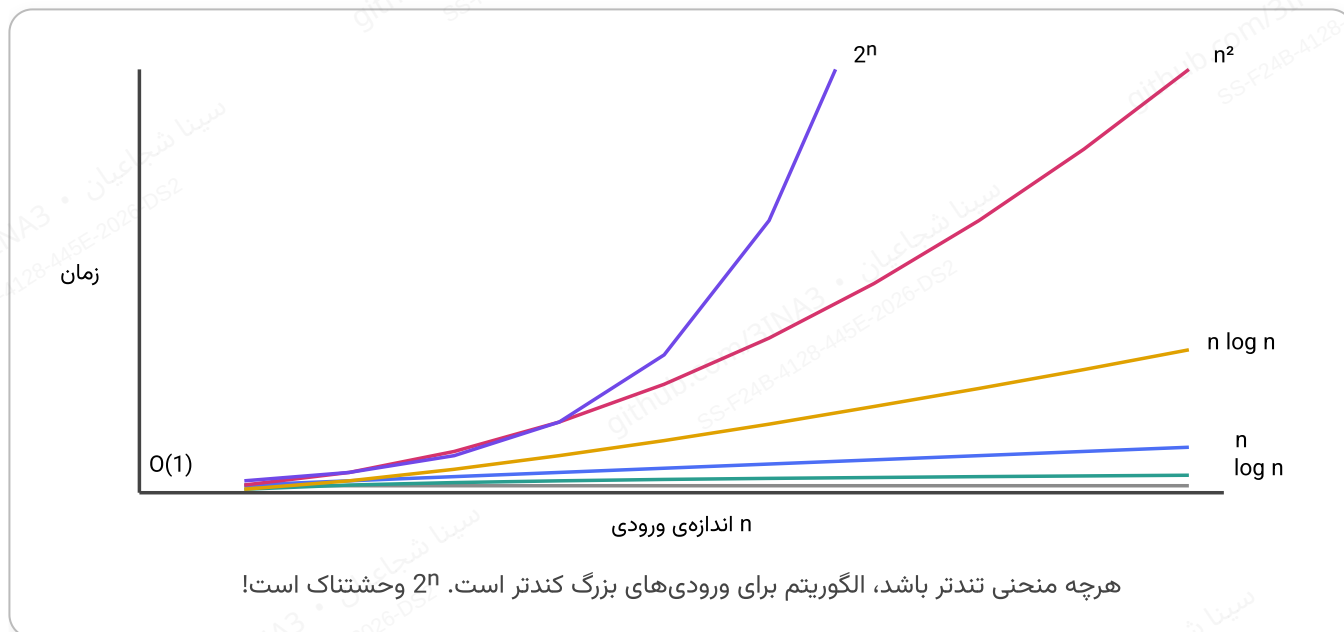
سریع‌تر پیدا می‌کنید (سریع). تحلیل الگوریتم یعنی بفهمیم «چقدر کار» لازم است تا مسئله حل شود وقتی ورودی بزرگ می‌شود.

وقتی برنامه‌ای می‌نویسیم، دو چیز مهم است:

- **زمان** – چند عملیات طول می‌کشد؟ (تابع پیچیدگی $f(n)$)
- **حافظه** – چقدر فضای اضافی لازم است؟

ما معمولاً فقط به «بدترین حالت» و «بزرگ‌ترین توان» نگاه می‌کنیم. مثلاً اگر $f(n) = 2n^2 + n + 1$ باشد، فقط n^2 مهم است و می‌گوییم $O(n^2)$.

نماد	معنا	رابطه با $f(n)$	مثال ساده
O (بزرگ‌او)	حد بالا – بدتر از این نمی‌شود	$f \leq c \cdot g$	مرتب‌سازی حبابی $O(n^2)$
Ω	حد پایین – بهتر از این نمی‌شود	$f \geq c \cdot g$	جست‌وجوی خطی حداقل $\Omega(n)$
Θ	حد دقیق (هم بالا هم پایین)	هر دو	جمع لیست $\Theta(n)$



مثال‌های روزمره به ترتیب سرعت

تابع	پیچیدگی	چرا؟	مثال واقعی
<code>arr[i]</code>	$O(1)$	مستقیم با اندیس می‌رود	باز کردن کشوی شماره‌ی ۳
<code>sum_list</code>	$O(n)$	یک بار از اول تا آخر می‌رود	جمع کردن نمرات کلاس
<code>binary_search</code>	$O(\log n)$	هر بار نصف می‌شود	پیدا کردن اسم در دفتر تلفن مرتب

merge_sort	$O(n \log n)$	$\log n$ سطح $\times n$ کار	مرتب کردن کارت‌های بازی با تقسیم
حبابی / درج	$O(n^2)$	حلقه‌ی تودرتو	مرتب کردن با مقایسه‌ی دو به دو

نکته‌ی طلایی: وقتی n خیلی بزرگ می‌شود (مثلاً یک میلیون)، تفاوت $O(n)$ و $O(n^2)$ مثل تفاوت چند ثانیه با چند ساعت است!

قدم به قدم: $f(n)$ چگونه ساخته می‌شود؟

برای هر دستور بپرسید: «چند بار اجرا می‌شود؟» بعد همه‌ی آن‌ها را جمع بزنید.

1 حلقه‌ی بیرونی n بار اجرا می‌شود.

2 حلقه‌ی درونی در هر دور بیرونی، n بار اجرا می‌شود $\rightarrow n \times n = n^2$

3 اگر داخل حلقه‌ی درونی ۲ دستور باشد $\rightarrow 2n^2$

4 دستورهای بیرون حلقه را هم اضافه کنید $\rightarrow 2n^2 + n + 1$

5 بزرگ‌ترین توان را نگه دارید و ضریب را دور بیندازید $\rightarrow \Theta(n^2)$

n	n	$n \log n$	n^2	2^n
10	10	≈ 33	100	1024
20	20	≈ 86	400	بیش از یک میلیون
1000	1000	≈ 9966	۱,۰۰۰,۰۰۰	غیرقابل تصور

حافظه: فقط فضای اضافی را حساب می‌کنیم. اگر فقط یک متغیر total داشته باشیم $\rightarrow O(1)$. اگر لیست کمکی بسازیم $\rightarrow O(n)$.

کدهای ساده با توضیح خط به خط

```
def worker(arr: list[int], index: int) → int:
    """0(1) → دسترسی مستقیم: کار به اندازه‌ی لیست بستگی ندارد"""
    return arr[index]          # فقط یک خواندن از حافظه

def linear_sum(arr: list[int]) → int:
    """0(n) → هر عنصر دقیقاً یک بار دیده می‌شود"""
    total = 0                  # 0(1) یک متغیر کمکی → حافظه
    for element in arr:        # بار اجرا می‌شود n حلقه
        total += element       # جمع کردن
    return total

if __name__ == "__main__":
    data = [4, 8, 15, 16]
    print(worker(data, 2), linear_sum(data))    # 15 43
```

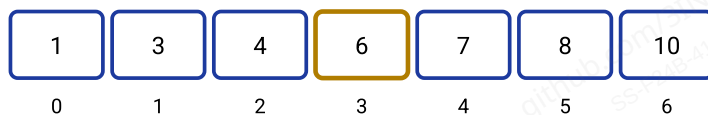
چرا این مهم است؟ اگر لیست شما ۱۰۰ میلیون عنصر داشته باشد، `worker` هنوز در یک لحظه جواب می‌دهد، اما `linear_sum` باید ۱۰۰ میلیون بار جمع بزند.

جست‌وجوی دودویی – نصف کردن هوشمندانه

شرط مهم: آرایه باید مرتب باشد. ایده‌ی ساده: وسط را نگاه کن. اگر هدف بزرگ‌تر بود، نیمه‌ی چپ را دور بریز. اگر کوچک‌تر بود، نیمه‌ی راست را دور بریز. هر بار بازه نصف می‌شود.

```
جست‌وجوی 7 در [10, 8, 7, 6, 4, 3, 1]
low=0 high=6
mid=3 → 6 < 7 → low=4      بازه: [10, 8, 7]
mid=5 → 8 > 7 → high=4     بازه: [7]
mid=4 → 7 == 7 → 4 اندیس پیدا شد!
```

!برای ۱ میلیون عنصر فقط حدود ۲۰ مقایسه لازم است



① $\text{mid}=3 \rightarrow 6 < 7 \rightarrow \text{low}=4$

② $\text{mid}=5 \rightarrow 8 > 7 \rightarrow \text{high}=4$

③ $\text{mid}=4 \rightarrow 7 == 7 \checkmark$

پیدا شد! اندیس ۴

مقایسه $\lceil \log_2 n \rceil$ هر مرحله بازه نصف می‌شود $\rightarrow$

جستجوی دودویی برای ۷: سه مرحله تا پیدا شدن

Python - binary_search

```
def binary_search(arr: list[int], target: int) -> int:
    """را برمی‌گرداند یا -1 اگر نبود target اندیس"""
    # حافظه: O(1)    زمان: O(log n)
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

if __name__ == "__main__":
    nums = [1, 3, 4, 6, 7, 8, 10]
    print(binary_search(nums, 7))    # 4
    print(binary_search(nums, 5))    # -1
```

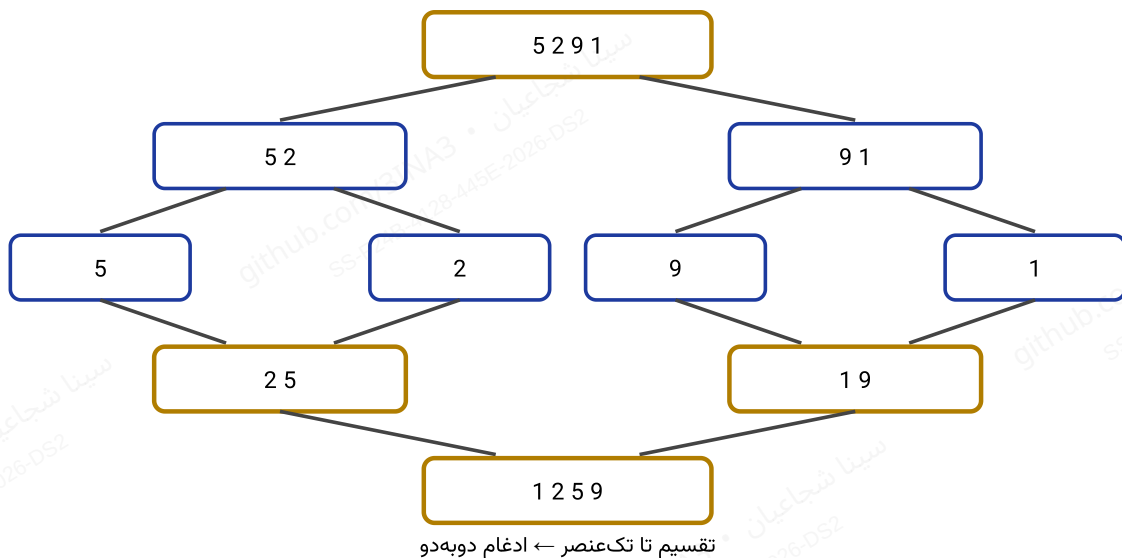
توضیح خط به خط binary_search:

1. $\text{low}, \text{high} = 0, \text{len}(\text{arr}) - 1$ - بازه‌ی جست‌وجو از اول تا آخر آرایه.
2. $\text{while low} \leq \text{high}$ - تا وقتی بازه حداقل یک عنصر دارد ادامه بده.
3. $\text{mid} = (\text{low} + \text{high}) // 2$ - اندیس وسط (تقسیم صحیح تا اعشار نیاید).
4. اگر $\text{arr}[\text{mid}] == \text{target}$ - پیدا شد، اندیس را برگردان.
5. اگر هدف بزرگ‌تر است $\rightarrow \text{low} = \text{mid} + 1$ (نیمه‌ی چپ را دور بریز).
6. اگر هدف کوچک‌تر است $\rightarrow \text{high} = \text{mid} - 1$ (نیمه‌ی راست را دور بریز).
7. اگر حلقه تمام شد و پیدا نشد $\rightarrow -1$.

مرتب‌سازی ادغامی (Merge Sort)

ایده: لیست را نصف کن، هر نیمه را مرتب کن، بعد دو لیست مرتب را با هم ادغام کن. این کار را آنقدر تکرار کن تا به لیست‌های یک‌عنصری برسی (که خودش مرتب است).

```
[5 2 9 1]
  /   \
[5 2] [9 1]   ← تقسیم
 \ /   \ /
[2 5] [1 9]   ← ادغام
  \ /
[1 2 5 9]
```



تقسیم تا تک‌عنصر ← ادغام دوبه‌دو

مراحل کامل Merge Sort: تقسیم از بالا به پایین، ادغام از پایین به بالا

```
def merge_sort(arr: list[int]) -> list[int]:
    """حافظه اضافی  $O(n)$ ، زمان  $O(n \log n)$  همیشه"""
    if len(arr) ≤ 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    # حالا دو لیست مرتب را ادغام می‌کنیم
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    return out + left[i:] + right[j:]

if __name__ == "__main__":
    print(merge_sort([5, 2, 9, 1, 7]))
```

توضیح خط به خط merge_sort:

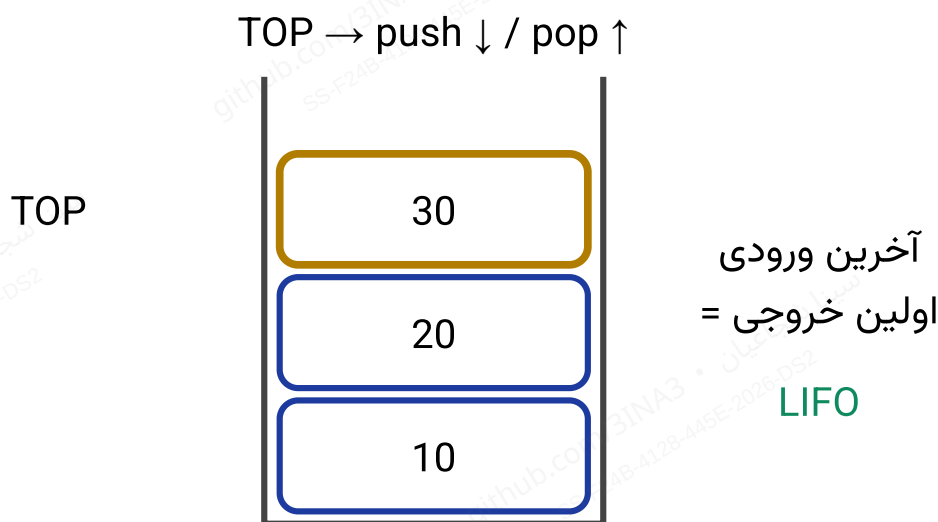
1. `if len(arr) ≤ 1` - اگر لیست خالی یا تک‌عنصری باشد، خودش مرتب است؛ برگردان.
2. `mid = len(arr) // 2` - نقطه‌ی وسط را پیدا کن (تقسیم صحیح).
3. `left = merge_sort(arr[:mid])` - نیمه‌ی چپ را بازگشتی مرتب کن.
4. `right = merge_sort(arr[mid:])` - نیمه‌ی راست را هم مرتب کن.
5. `out, i, j = [], 0, 0` - لیست نتیجه و دو اشاره‌گر برای پیمایش `left` و `right`.
6. حلقه‌ی `while`: همیشه عنصر کوچک‌تر را به `out` اضافه کن و اشاره‌گرش را جلو ببر.
7. `return out + left[i:] + right[j:]` - هر کدام که تمام نشده، باقی‌مانده‌اش را بچسبان.

۲. پشته (Stack) – آخرین ورودی، اولین خروجی

آنالوژی روزمره: یک بسته‌ی بشقاب را تصور کنید. بشقاب جدید را روی بقیه می‌گذارید (push). وقتی بشقاب می‌خواهید، از بالا برمی‌دارید (pop). هیچ‌وقت از وسط یا پایین بر نمی‌دارید. این همان **LIFO** است: Last In, First Out.

پشته فقط از یک طرف (بالا / TOP) کار می‌کند. دو عملیات اصلی:

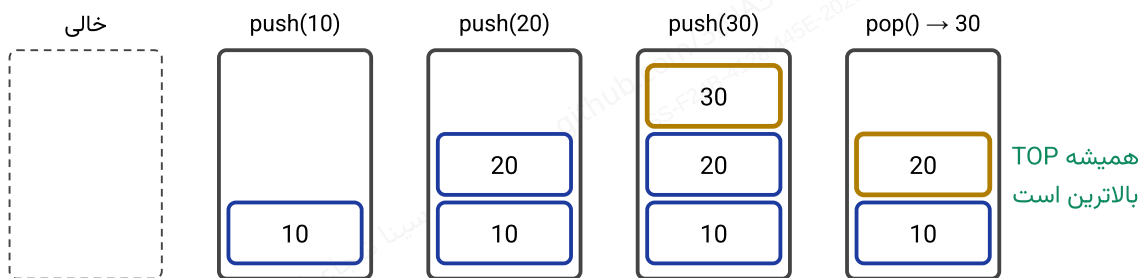
- $\text{push}(x)$ – عنصر را روی پشته بگذار (شرط: پر نباشد $\rightarrow$ Overflow)
- $\text{pop}()$ – عنصر بالای پشته را بردار (شرط: خالی نباشد $\rightarrow$ Underflow)
- $\text{peek}()$ – فقط ببین بالای پشته چیست، بدون برداشتن



ته پشته (اولین ورودی)

پشته: ورود و خروج فقط از بالا. آخرین چیزی که گذاشتی، اولین چیزی است که برمی‌داری.

```
push(1)  $\rightarrow$  [1]
push(2)  $\rightarrow$  [1, 2]   سمت راست TOP
push(3)  $\rightarrow$  [1, 2, 3]
pop()     $\rightarrow$  3        حالا [2, 1]
(ثابت - به اندازه‌ی پشته بستگی ندارد)  $O(1)$ : همه‌ی عملیات
```



مراحل push و pop: هر push یک لایه بالا می‌آورد، هر pop لایه بالا را برمی‌دارد

```
class Stack:
    """پایتون list پشته با ظرفیت محدود - ساده‌ترین پیاده‌سازی با"""

    def __init__(self, limit: int = 100) → None:
        self.items: list = []      # لیست خالی برای نگهداری عناصر
        self.limit = limit         # حداکثر ظرفیت

    def is_empty(self) → bool:
        return not self.items      # True اگر لیست خالی باشد

    def push(self, x) → bool:
        if len(self.items) ≥ self.limit:  # اگر پر بود
            return False                  # جا نیست - Overflow
        self.items.append(x)              # به انتهای لیست اضافه کن
        return True

    def pop(self):
        if self.is_empty():               # اگر خالی بود
            raise IndexError("stack is empty")  # Underflow
        return self.items.pop()           # آخرین عنصر را بردار

    def peek(self):
        if self.is_empty():
            raise IndexError("stack is empty")
        return self.items[-1]             # فقط نگاه کن، برنदार

    def show(self) → None:
        print(*reversed(self.items))      # به پایین چاپ کن TOP از

if __name__ == "__main__":
    s = Stack()
    s.push(10); s.push(20); s.push(30)
    s.show()                             # 30 20 10
    print(s.peek())                       # 30
    print(s.pop())                         # 30
    s.show()                              # 20 10
```

توضیح خط‌به‌خط کلاس Stack:

1. `self.items = []` - یک لیست خالی پایتون؛ انتهای لیست = TOP پشته.
2. `is_empty` - اگر لیست خالی باشد، پشته خالی است.

3. push – اول ظرفیت را چک می‌کند (Overflow). بعد با append به انتها اضافه می‌کند $O(1)$.
4. pop – اول خالی بودن را چک می‌کند (Underflow). بعد با `list.pop()` آخرین عنصر را برمی‌دارد $O(1)$.
5. peek – فقط `items[-1]` را برمی‌گرداند؛ لیست تغییر نمی‌کند.
6. show – با `reversed` از بالا به پایین چاپ می‌کند تا شبیه پشته واقعی باشد.

کاربردهای واقعی پشته

1. دکمه‌ی Undo در ویرایشگر متن – آخرین تغییری که دادی، اول برمی‌گردد.
2. Call Stack – وقتی تابعی تابع دیگری را صدا می‌زند، آدرس بازگشت در پشته ذخیره می‌شود.
3. بررسی پرانتزها – هر پرانتز باز را push کن، هر بسته‌ای که آمد باید با آخرین باز جور باشد.
4. تبدیل مبنا – باقی‌مانده‌ها به ترتیب معکوس تولید می‌شوند، پشته آن‌ها را درست می‌چیند.

مثال کامل: تبدیل عدد دهدهی به دودویی

باقی‌مانده‌های تقسیم بر ۲ از کم‌ارزش به پرارزش تولید می‌شوند، اما ما باید از پرارزش شروع کنیم. پشته ترتیب را برعکس می‌کند.

n	n % 2 (push)	n // 2	پشته (TOP راست)
13	1	6	[1]
6	0	3	[0, 1]
3	1	1	[1, 0, 1]
1	1	0	[1, 1, 0, 1]

حالا چهار بار $1101 \Rightarrow 1 \leftarrow 1 \leftarrow 0 \leftarrow 1$: pop

```
def d2b(n: int) → str:
    """تبدیل دهدهی به دودویی با پشته"""
    if n == 0:
        return "0"
    s = Stack()
    while n:
        s.push(n % 2)          # کن push باقیمانده را
        n //= 2                 # خارج قسمت را ادامه بده
    out = ""
    while not s.is_empty():
        out += str(s.pop())     # از بالا بخوان → ترتیب درست
    return out

print(d2b(13))    # 1101
print(d2b(7))     # 111
print(d2b(0))     # 0
```

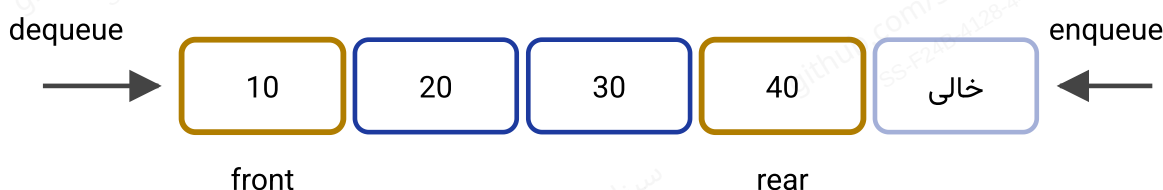
بررسی تعادل پرانتزها

```
def balanced(text: str) → bool:
    """آیا همه‌ی پرانتزها، براکت‌ها و آکولادها درست جفت شده‌اند؟"""
    pairs = {"(": ")", "[": "]", "{": "}"} # بسته‌ها → بازها
    s = Stack()
    for ch in text:
        if ch in "([{":
            s.push(ch) # پرانتز باز
        elif ch in pairs:
            # پرانتز بسته
            if s.is_empty() or s.pop() != pairs[ch]:
                return False # جفت نبود یا پشته خالی
    return s.is_empty() # در پایان پشته باید خالی باشد

print(balanced("{[()]}" )) # True
print(balanced("[ ]") ) # False
print(balanced("(a+b)") ) # False (یک باز اضافی)
```

۳. صف و صف حلقوی – اولین ورودی، اولین خروجی

آنالوژی: صف نانوائی. کسی که اول آمده، اول نان می‌گیرد. کسی که آخر آمده، آخر می‌گیرد. این **FIFO** است: First In, First Out. درج از انتهای صف (rear)، حذف از ابتدای صف (front).



FIFO: اول وارد → اول خارج
صف خطی: درج از rear، حذف از front

مشکل صف خطی: با هر حذف، خانه‌ی اول هدر می‌رود و دیگر استفاده نمی‌شود. وقتی rear به آخر آرایه برسد، حتی اگر اول صف خالی باشد، صف «پر» اعلام می‌شود.

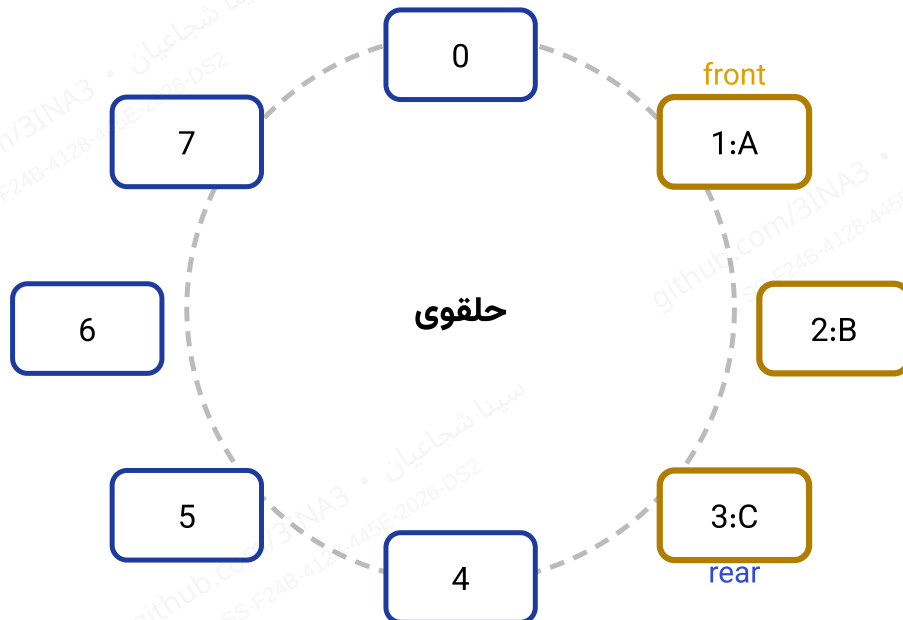
```
limit=5
enqueue(10), (20), (30)    dequeue()→10
[10][20][30][ ][ ]      [ ][20][30][ ][ ]
↑f      ↑r              ↑f  ↑r
!خانه‌ی ۰ هدر رفت و دیگر استفاده نمی‌شود
```

صف حلقوی – راه‌حل هوشمندانه

با عملکرد باقی‌مانده (%) وقتی به آخر آرایه رسیدیم، به ابتدا برمی‌گردیم (Wrap Around). حالا فضای خالی اول صف دوباره قابل استفاده است.

شرط صف حلقوی	حالت
$front == -1$	خالی
$(rear + 1) \% limit == front$	پر
$front == rear$	تک‌عنصری

اندیس: 0 1 2 3 4 5 6 7 limit=8
 مقدار: [50] [] [] [] [] [20] [30] [40]
 $\uparrow f$ $\uparrow r$
 بعدی = $(1+7) \% 8 = 0 \leftarrow$ بازگشت به ابتدا rear



insert(D): front=2, rear=0 (wrap) و delete(A) بعد از

نمای دایره‌ای صف حلقوی: بعد از پر شدن، rear به خانه‌ی ۰ برمی‌گردد

```

class CQueue:
    """صف حلقوی با ظرفیت ثابت."""

    def __init__(self, limit: int = 10) → None:
        self.q = [None] * limit          # آرایه‌ی خالی با اندازه‌ی ثابت
        self.limit = limit
        self.front = self.rear = -1      # هر دو -1 یعنی صف خالی

    def is_empty(self) → bool:
        return self.front == -1

    def is_full(self) → bool:
        # باشد → پر front همان rear یک خانه جلوتر از
        return (self.rear + 1) % self.limit == self.front

    def insert(self, x) → None:          # enqueue
        if self.is_full():
            raise OverflowError("queue is full")
        if self.is_empty():              # اولین عنصر
            self.front = self.rear = 0
        else:
            self.rear = (self.rear + 1) % self.limit # !حلقوی
        self.q[self.rear] = x

    def delete(self):                    # dequeue
        if self.is_empty():
            raise IndexError("queue is empty")
        x = self.q[self.front]
        if self.front == self.rear:      # آخرین عنصر بود → خالی شود
            self.front = self.rear = -1
        else:
            self.front = (self.front + 1) % self.limit
        return x

    def display(self) → None:
        if self.is_empty():
            print("empty"); return
        i = self.front
        while True:
            print(self.q[i], end=" ")
            if i == self.rear:
                break
            i = (i + 1) % self.limit

```

```
print()
```

```
if __name__ == "__main__":  
    q = CQueue(3)  
    q.insert(1); q.insert(2); q.insert(3)  
    q.delete()           # خارج شد 1  
    q.insert(4)          # حالا جا برای 4 هست  
    q.display()          # 2 3 4
```

توضیح خطبه خط صف حلقوی:

1. `self.q = [None] * limit` - آرایه‌ی ثابت‌اندازه؛ خانه‌های خالی `None` هستند.
2. `front = rear = -1` - نشانه‌ی «صف خالی». با اولین `insert` هر دو ۰ می‌شوند.
3. `is_full` - اگر `(rear+1) % limit == front` باشد، یک خانه خالی هم نمانده (عمداً یک خانه را خالی نگه می‌داریم تا پر و خالی قاطی نشوند).
4. `insert - rear` را با `%` جلو می‌برد (حلقوی) و مقدار را می‌نویسد.
5. `delete` - مقدار `front` را برمی‌دارد، بعد `front` را با `%` جلو می‌برد. اگر `front==rear` بود، صف خالی می‌شود.
6. `display` - از `front` شروع می‌کند و با `%` دور می‌زند تا به `rear` برسد.

تمرین ذهنی: صف حلقوی با ظرفیت ۳ بسازید. ترتیب عملیات: `insert(A), insert(B), delete, insert(C), insert(D)`. وضعیت نهایی چیست؟ (پاسخ: B C D)

۴. توابع بازگشتی – تابعی که خودش را صدا می‌زند

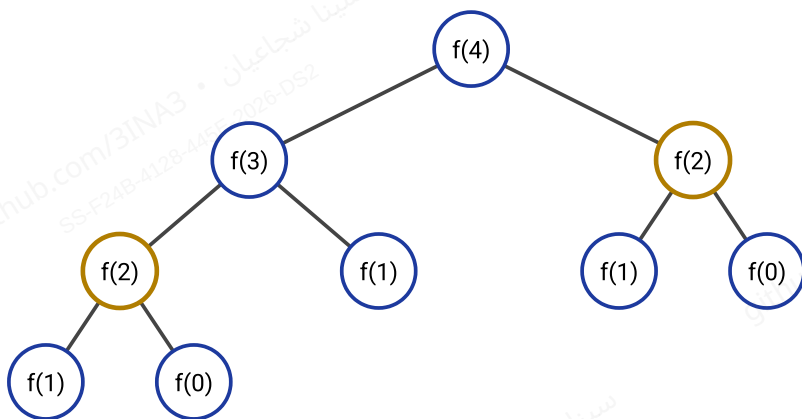
آنالوژی: تصور کنید می‌خواهید یک جعبه‌ی تو در تو را باز کنید. هر جعبه داخلش جعبه‌ی کوچک‌تری دارد. شما جعبه را باز می‌کنید، جعبه‌ی داخل را به دوستتان می‌دهید تا او باز کند، و همین‌طور ادامه می‌دهید تا به آخرین جعبه‌ی خالی برسید. بعد جواب‌ها یکی‌یکی برمی‌گردند. این همان **بازگشت (Recursion)** است.

هر تابع بازگشتی سه جزء دارد:

1. **شرط توقف (Base Case)** – کی باید تمام شود؟ (وگرنه بی‌نهایت صدا می‌زند و برنامه می‌ترکد)
2. **مقدار بازگشت** – وقتی شرط توقف برقرار شد، چه چیزی برگردانیم؟
3. **رابطه‌ی بازگشتی** – مسئله را به مسئله‌ی کوچک‌تر تبدیل کن.

هشدار: اگر شرط توقف ننویسید یا هیچ‌وقت برقرار نشود، پایتون با خطای `RecursionError` متوقف می‌شود (پیش‌فرض حدود ۱۰۰۰ فراخوانی).

تابع	شرط توقف	رابطه	زمان
فاکتوریل	$n \leq 1 \rightarrow 1$	$n! = n \times (n-1)$	$O(n)$
فیبوناچی	$n < 2 \rightarrow n$	$f(n) = f(n-1) + f(n-2)$	$O(2^n)$ بدون حافظه
جمع $a+b$	$b=0 \rightarrow a$	$S(a,b) = S(a,b-1) + 1$	$O(b)$
برج هانوی	$n=1 \rightarrow$ یک حرکت	$T(n) = 2T(n-1) + 1$	$O(2^n)$

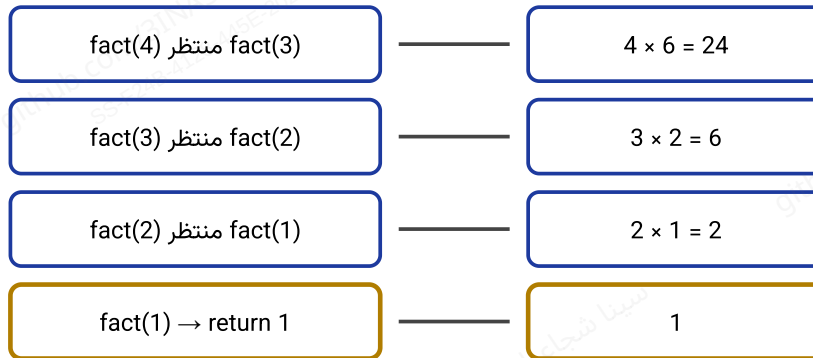


دوبار حساب می‌شود $f(2)$
(رنگ نارنجی)

درخت فراخوانی $fib(4)$: زیرمسئله‌های تکراری دلیل کندی فیبوناچی ساده هستند.

ردگیری Call Stack (LIFO) روی $\text{fact}(4)$:

برگشت (جواب)	رفت (فراخوانی)
$\text{fact}(4) = 4 * \text{fact}(3)$	$\text{fact}(1) = 1$
$\text{fact}(3) = 3 * \text{fact}(2)$	$\text{fact}(2) = 2 * 1 = 2$
$\text{fact}(2) = 2 * \text{fact}(1)$	$\text{fact}(3) = 3 * 2 = 6$
$\text{fact}(1) \rightarrow$ شرط توقف	$\text{fact}(4) = 4 * 6 = 24$



هر قاب تا رسیدن
جواب زیرفراخوانی
منتظر می ماند

Call Stack (از بالا به پایین رشد می کند)

برگشت جوابها (از پایین به بالا)

Call Stack برای $\text{fact}(4)$: فراخوانی ها روی هم انباشته می شوند، بعد جوابها از پایین برمی گردند

```

def fact(n: int) → int:
    """فاکتوریل:  $n! = n \times (n-1)!$ """
    if n ≤ 1:
        # شرط توقف
        return 1
    return n * fact(n - 1) # رابطه‌ی بازگشتی

def fib(n: int) → int:
    """بزرگ n فیبوناچی ساده - کند برای"""
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

def fib_memo(n: int, memo: dict | None = None) → int:
    """(Memoization) →  $O(n)$  فیبوناچی با حافظه"""
    if memo is None:
        memo = {}
    if n < 2:
        return n
    if n not in memo:
        # قبلاً حساب نشده؟
        memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]

def is_palindrome(s: str) → bool:
    """آیا رشته از دو طرف یکسان خوانده می‌شود؟"""
    if len(s) ≤ 1:
        # شرط توقف
        return True
    return s[0] == s[-1] and is_palindrome(s[1:-1])

def hanoi(n: int, src: str, aux: str, dst: str) → None:
    """برج هانوی: n دیسک از src به dst با کمک aux"""
    if n == 1:
        print(src, "→", dst)
        return
    hanoi(n-1, src, dst, aux) # n-1 به کمکی ببر
    print(src, "→", dst) # بزرگ‌ترین را جابه‌جا کن
    hanoi(n-1, aux, src, dst) # n-1 روی مقصد بگذار

if __name__ == "__main__":

```

```
print(fact(6))          # 720
print(fib_memo(40))     # 102334155 (!سريع)
print(is_palindrome("radar")) # True
hanoi(2, "A", "B", "C")
# A → B
# A → C
# B → C
```

توضیح خطبه خط fact و fib_memo:

1. `if n <= 1: return 1` - شرط توقف حیاتی. بدون آن برنامه تا بی نهایت صدا می زند.
2. `return n * fact(n-1)` - مسئله را به یک مسئله کوچک تر تبدیل می کند و منتظر جواب می ماند.
3. در `fib_memo`: دیکشنری `memo` نتایج قبلی را نگه می دارد.
4. `if n not in memo` - فقط اگر قبلاً حساب نشده، محاسبه کن و ذخیره کن.
5. نتیجه: به جای $O(2^n)$ فقط $O(n)$ فراخوانی داریم.

چرا فیبوناچی ساده کند است؟ چون زیرمسئله های تکراری را بارها حساب می کند. با Memoization هر مقدار فقط یک بار محاسبه می شود و نتیجه ذخیره می شود.

۵. لیست پیوندی – زنجیره‌ای از نودها

آنالوژی: یک قطار را تصور کنید. هر واگن (نود) هم محتوای خودش را دارد (مسافر/داده) و هم به واگن بعدی وصل است (اشاره‌گر next). برخلاف آرایه که همه‌ی خانه‌ها کنار هم در حافظه‌اند، اینجا هر نود می‌تواند هر جای حافظه باشد و فقط با اشاره‌گر به هم وصل‌اند.



next یک‌طرفه: فقط



next و back: دوطرفه

بالا: لیست یک‌طرفه – پایین: لیست دوطرفه

یک‌طرفه: HEAD → [10 | •] → [20 | •] → [30 | •] → None
حلقوی: HEAD → [10 | •] → [20 | •] → [30 | •] → HEAD
دوطرفه: None ⇌ [10] ⇌ [20] ⇌ [30] ⇌ None

insert_first(5) قبل از



- ۱) n.next = head
- ۲) head = n

insert_first(5) بعد از



درج در ابتدا: فقط دو اشاره‌گر عوض می‌شود → O(1)

مزیت و عیب نسبت به آرایه

عمل	آرایه	لیست پیوندی
-----	-------	-------------

$O(n)$	$O(1)$	دسترسی با اندیس
$O(1)$	$O(n)$ (جابه‌جایی)	درج / حذف ابتدا
$O(n)$ جست‌وجو + $O(1)$ تغییر اشاره‌گر	$O(n)$	درج / حذف وسط
پراکنده + اشاره‌گر اضافی	پیوسته	حافظه

```
from typing import Any, Optional
```

```
class Node:
```

```
    def __init__(self, data: Any) → None:
        self.data = data          # محتوای نود
        self.next = None          # اشاره‌گر به نود بعدی
```

```
class LinkedList:
```

```
    def __init__(self) → None:
        self.head: Optional[Node] = None    # شروع زنجیره
```

```
    def insert_first(self, x) → None:
```

```
        """O(1) - درج در ابتدا"""
        n = Node(x)
        n.next = self.head    # نود جدید به سر قبلی اشاره کند
        self.head = n        # سر جدید شود
```

```
    def insert_last(self, x) → None:
```

```
        """O(n) - درج در انتها"""
        n = Node(x)
        if not self.head:    # لیست خالی بود
            self.head = n
            return
        t = self.head
        while t.next:        # تا آخر برو
            t = t.next
        t.next = n          # آخری به نود جدید اشاره کند
```

```
    def delete_first(self) → None:
```

```
        if self.head:
            self.head = self.head.next    # سر را یکی جلو ببر
```

```
    def show(self) → None:
```

```
        t, out = self.head, []
        while t:
            out.append(str(t.data))
            t = t.next
        print(" → ".join(out) + " → None")
```

```
if __name__ == "__main__":
```

```
    L = LinkedList()
```

```
L.insert_last(20)
L.insert_last(30)
L.insert_first(10)
L.show()    # 10 → 20 → 30 → None
L.delete_first()
L.show()    # 20 → 30 → None
```

توضیح خطبه خط LinkedList:

1. Node - هر نود دو فیلد دارد: data (محتوا) و next (آدرس نود بعدی).
2. self.head - فقط آدرس اولین نود را نگه می‌داریم. بقیه از طریق زنجیره‌ی next پیدا می‌شوند.
3. insert_first - نود جدید می‌سازد، n.next را به head فعلی وصل می‌کند، بعد head را عوض می‌کند. دو خط $\rightarrow O(1)$.
4. insert_last - باید تا آخر لیست برود (while t.next)، بعد وصل کند $\rightarrow O(n)$.
5. delete_first - فقط head را یکی جلو می‌برد. نود قبلی دیگر قابل دسترس نیست و زباله‌روب آزادش می‌کند.
6. show - از head شروع می‌کند و تا رسیدن به None پیمایش می‌کند.

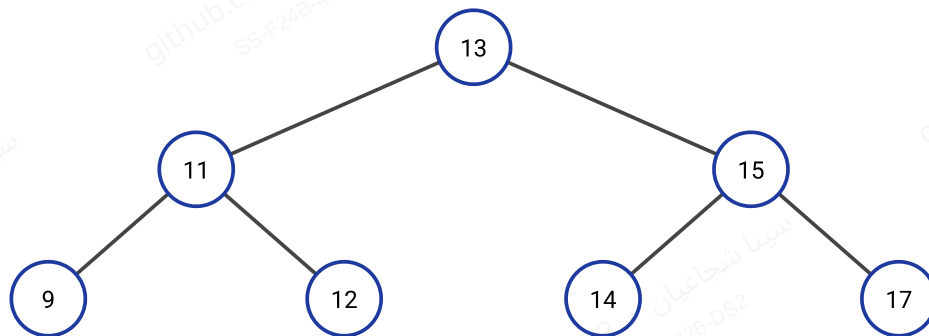
ترتیب طلایی اشاره‌گرها: همیشه اول `new.next = t.next` را تنظیم کنید، بعد `t.next = new`. اگر برعکس کنید، بقیه‌ی لیست گم می‌شود!

۶. درخت – ساختار سلسله‌مراتبی

آنالوژی: شجره‌نامه‌ی خانوادگی. یک نفر در رأس (ریشه / Root)، فرزندان، نوه‌ها و ... هر کس فقط یک پدر دارد (به جز ریشه). برگ‌ها کسانی هستند که فرزندی ندارند.

مفاهیم کلیدی:

- ریشه (Root) – بالای درخت
- برگ (Leaf) – نود بدون فرزند
- درجه‌ی نود – تعداد فرزندان
- ارتفاع – طولانی‌ترین مسیر از ریشه تا یک برگ
- درخت دودویی – هر نود حداکثر ۲ فرزند دارد



NLR (پیش‌ترتیب): 17 14 15 12 9 11 13

LNR (میان‌ترتیب): 17 15 14 13 12 11 9

LRN (پس‌ترتیب): 13 15 17 14 11 12 9

سطحی (BFS): 13 11 15 9 12 14 17

یک درخت دودویی و چهار روش پیمایش

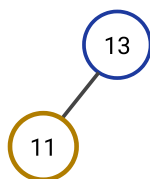
درخت جست‌وجوی دودویی (BST)

قانون طلایی: چپ > پدر > راست. این قانون باعث می‌شود جست‌وجو، درج و حذف در حالت متوسط $O(\log n)$ باشد.

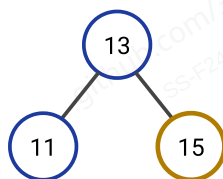
add(13)



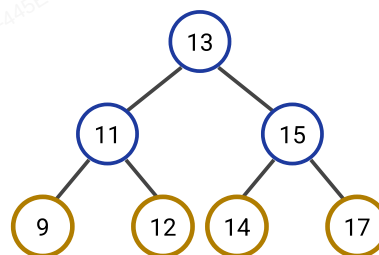
add(11)



add(15)



add(9) ... add(17)



هر عدد جدید از ریشه شروع می‌کند و با مقایسه چپ/راست جای خالی پیدا می‌کند

چپ | $9 > 11$ → چپ → برگ | $13 < 17$ → راست | $15 < 17$ → راست → برگ | $9 < 13$ → برگ

ساخت BST با درج متوالی ۱۷، ۱۴، ۱۲، ۹، ۱۵، ۱۱، ۱۳

```

class BNode:
    def __init__(self, d) → None:
        self.data = d
        self.left = None
        self.right = None

class BST:
    def __init__(self) → None:
        self.root = None

    def add(self, x) → None:
        if not self.root:
            self.root = BNode(x)
            return
        t = self.root
        while True:
            if x == t.data:
                return # تکراری قبول نمی‌کنیم
            if x < t.data:
                if t.left is None:
                    t.left = BNode(x)
                    return
                t = t.left
            else:
                if t.right is None:
                    t.right = BNode(x)
                    return
                t = t.right

    def search(self, x) → bool:
        t = self.root
        while t and t.data ≠ x:
            t = t.left if x < t.data else t.right
        return t is not None

    def inorder(self) → list:
        """میان‌ترتیب همیشه لیست مرتب می‌دهد"""
        result = []
        def walk(n):
            if n:
                walk(n.left)
                result.append(n.data)

```

```

        walk(n.right)
    walk(self.root)
    return result

```

```

if __name__ == "__main__":
    b = BST()
    for v in (13, 11, 15, 9, 12, 14, 17):
        b.add(v)
    print(b.inorder())      # [9, 11, 12, 13, 14, 15, 17]
    print(b.search(14))     # True
    print(b.search(99))     # False

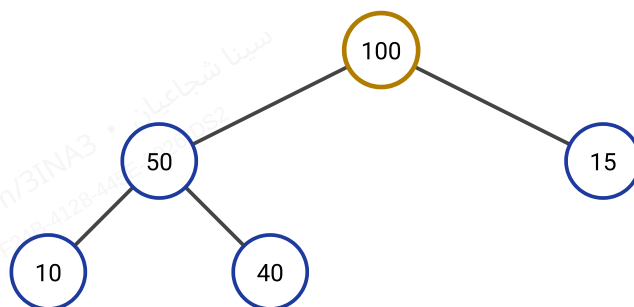
```

توضیح خطبه خط BST.add:

1. اگر ریشه None باشد، نود جدید ریشه می‌شود.
2. از ریشه شروع می‌کنیم و در یک حلقه بی‌نهایت (while True) حرکت می‌کنیم.
3. اگر $x == t.data$ - تکراری است، کاری نمی‌کنیم.
4. اگر $x < t.data$ - باید به چپ برویم. اگر چپ خالی بود، همان‌جا درج کن؛ وگرنه $t = t.left$.
5. اگر $x > t.data$ - همین منطق برای راست.
6. جست‌وجو دقیقاً همین مسیر را می‌رود؛ اگر به None رسید یعنی نبود.

هیپ (Heap) - درخت کامل با اولویت

در Max-Heap هر پدر از فرزندانش بزرگ‌تر است. ریشه همیشه بزرگ‌ترین عنصر است. در لیست ذخیره می‌شود: فرزندان نود i در خانه‌های $2i+1$ و $2i+2$.



فرزندان i : $2i+1$ و $2i+2$
پدر: $(i-1)//2$

لیست:

100	50	15	10	40
0	1	2	3	4

Max-Heap: درخت کامل که در یک لیست ساده ذخیره می‌شود

```

class MaxHeap:
    def __init__(self) → None:
        self.a: list = []

    def insert(self, x) → None:
        self.a.append(x)          # به انتها اضافه کن
        self._up(len(self.a) - 1) # به بالا حباب بزن

    def _up(self, n: int) → None:
        while n > 0:
            p = (n - 1) // 2      # پدر
            if self.a[n] ≤ self.a[p]:
                break
            self.a[n], self.a[p] = self.a[p], self.a[n]
            n = p

    def del_root(self):
        if not self.a:
            raise IndexError("heap is empty")
        top = self.a[0]
        last = self.a.pop()
        if self.a:
            self.a[0] = last
            self._down(0)         # به پایین حباب بزن
        return top

    def _down(self, n: int) → None:
        while True:
            big = n
            l, r = 2*n+1, 2*n+2
            if l < len(self.a) and self.a[l] > self.a[big]:
                big = l
            if r < len(self.a) and self.a[r] > self.a[big]:
                big = r
            if big == n:
                return
            self.a[n], self.a[big] = self.a[big], self.a[n]
            n = big

if __name__ == "__main__":
    h = MaxHeap()
    for v in (10, 40, 15, 50, 100):

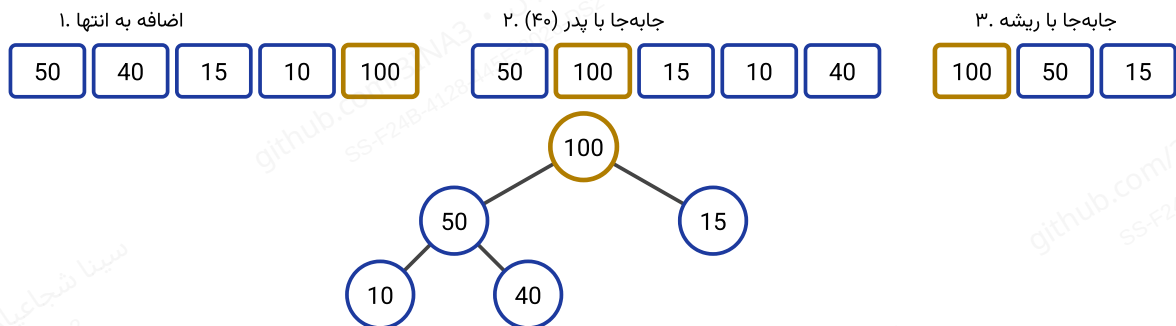
```

```
h.insert(v)
print([h.del_root() for _ in range(5)])
# [100, 50, 40, 15, 10] ← همیشه بزرگ‌ترین خارج می‌شود
```

توضیح خطبه خط MaxHeap:

1. insert - عنصر را به انتهای لیست اضافه می‌کند، بعد $_up$ را صدا می‌زند.
2. $_up$ (bubble up) - تا وقتی پدر کوچک‌تر باشد، با پدر جابه‌جا می‌شود. اندیس پدر: $(i-1)//2$.
3. $_del_root$ - ریشه (بزرگ‌ترین) را برمی‌دارد، آخرین عنصر را جای ریشه می‌گذارد، بعد $_down$.
4. $_down$ (bubble down) - با بزرگ‌ترین فرزند جابه‌جا می‌شود تا ترتیب هیپ درست شود. فرزندان: $2i+1$ و $2i+2$.
5. هر دو عملیات $O(\log n)$ هستند چون ارتفاع درخت کامل حدود $\log n$ است.

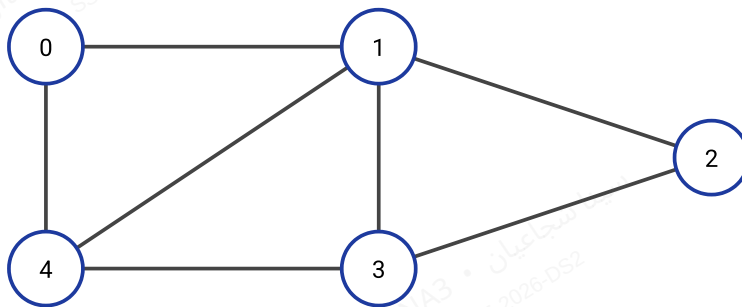
گام به گام bubble up: درج ۱۰۰



درج ۱۰۰ در Max-Heap: از انتها اضافه می‌شود و با $_bubble_up$ به ریشه می‌رسد

۷. گراف – شبکه‌ی رئوس و یال‌ها

آنالوژی: نقشه‌ی شهرها و جاده‌ها. هر شهر یک رأس (Vertex)، هر جاده یک یال (Edge). اگر جاده یک‌طرفه باشد، گراف جهت‌دار است. اگر وزن (فاصله) داشته باشد، وزن‌دار است.



ماتریس مجاورت

	0	1	2	3	4
0	0	1	0	0	1
1	1	1	0	1	1
2	0	1	0	1	0
3	0	1	1	0	1
4	1	1	0	1	0

گراف بدون جهت با ۵ رأس و ۷ یال + ماتریس مجاورت

BFS (صف) – سطح‌به‌سطح

از 0: 0 → 1 → 2 → 3 → 4

DFS (عمق) – (پشته/بازگشت)

از 0: 0 → 1 → 4 → 3 → 2

BFS (صف)

0 → [1,4]: صف

1 → [2,3]: صف

4 → [2,3]: صف

2 → [3]: صف

3 خروجی: 0 1 2 3 4

DFS (عمق‌اول)

0 → 1 برو به

1 → 2 برو به

2 → 3 برو به

3 → 4 برو به

4 خروجی: 0 1 2 3 4

BFS سطح‌به‌سطح با صف جلو می‌رود؛ DFS تا عمق می‌رود و بعد برمی‌گردد

```

from collections import deque

class Graph:
    def __init__(self, n: int, directed: bool = False) → None:
        self.n = n
        self.directed = directed
        self.m = [[0]*n for _ in range(n)] # ماتریس n×n

    def insert_edge(self, s: int, t: int, w: int = 1) → None:
        self.m[s][t] = w
        if not self.directed:
            self.m[t][s] = w # دوطرفه

    def bfs(self, start: int) → list[int]:
        """پیمایش سطح به سطح با صف"""
        seen, q, order = {start}, deque([start]), []
        while q:
            u = q.popleft()
            order.append(u)
            for v in range(self.n):
                if self.m[u][v] and v not in seen:
                    seen.add(v)
                    q.append(v)
        return order

    def dfs(self, start: int, seen: set | None = None) → list[int]:
        """پیمایش عمقی با بازگشت"""
        if seen is None:
            seen = set()
        seen.add(start)
        order = [start]
        for v in range(self.n):
            if self.m[start][v] and v not in seen:
                order += self.dfs(v, seen)
        return order

if __name__ == "__main__":
    g = Graph(5)
    for s, t in [(0,1),(0,4),(1,2),(1,3),(1,4),(2,3),(3,4)]:
        g.insert_edge(s, t)

```

```
print(g.bfs(0)) # [0, 1, 4, 2, 3]
print(g.dfs(0)) # [0, 1, 2, 3, 4]
```

توضیح خطبه خط BFS و DFS:

1. `seen` - مجموعه‌ی رأس‌های دیده‌شده تا دوباره نرویم (جلوگیری از حلقه).
2. **BFS**: صف (`deque`). اول رأس را از صف برمی‌داریم، بعد همه‌ی همسایه‌های ندیده را به صف اضافه می‌کنیم → سطح به سطح.
3. **DFS**: بازگشت (یا پشته). رأس فعلی را علامت می‌زنیم، بعد اولین همسایه‌ی ندیده را صدا می‌زنیم و تا عمق می‌رویم.
4. ماتریس مجاورت: $m[u][v] \neq 0$ یعنی یال وجود دارد.
5. زمان با ماتریس: $O(V^2)$ چون برای هر رأس همه‌ی ستون‌ها را چک می‌کنیم.

ویژگی	ماتریس مجاورت	لیست مجاورت
حافظه	$O(V^2)$	$O(V+E)$
آیا یال (s,t) هست؟	$O(1)$	$O(\text{درجه‌ی } s)$
BFS / DFS	$O(V^2)$	$O(V+E)$
مناسب برای	گراف انبوه	گراف خلوت

جدول مرور سریع و نکات امتحانی

نکته‌ی کلیدی	زمان	ساختار / عمل
Underflow و LIFO – Overflow را چک کن	$O(1)$	Stack push/pop/peek
FIFO – حلقوی با %	$O(1)$	Queue insert/delete
درج ابتدا $O(1)$	$O(n)$	Linked list جست‌وجو
درخت کج $\rightarrow$ بدترین حالت	$O(\log n) / O(n)$	BST جست‌وجو / درج
ریشه همیشه min یا max	$O(\log n)$	Heap insert / del root
صف / پشته	$O(V^2)$	BFS / DFS (ماتریس)
آرایه باید مرتب باشد	$O(\log n)$	binary_search
همیشه – بهترین و بدترین یکسان	$O(n \log n)$	merge_sort

- همیشه قبل از pop / delete / peek خالی بودن را چک کنید.
- در لیست پیوندی ترتیب بازنویسی اشاره‌گرها را حفظ کنید (اول new.next، بعد t.next).
- برای بازگشت: **اول شرط توقف** را بنویسید.
- در BST قانون چپ > پدر > راست را فراموش نکنید.
- در هیپ فقط رابطه‌ی پدر-فرزند مهم است؛ بین دو برادر ترتیبی نیست.

جمع‌بندی نهایی: ساختمان داده ابزار است. مثل جعبه‌ابزار: برای هر کار ابزار مناسب را انتخاب کنید. آرایه برای دسترسی سریع، پشته برای Undo، صف برای نوبت، درخت برای سلسله‌مراتب، گراف برای شبکه. هرچه بیشتر تمرین کنید، انتخاب ابزار برایتان غریزی می‌شود.